

Recolección de datos con Open MPI

Universidad del Valle de Guatemala · CC3069
Ejercicio 30 septiembre · Fernando Mendoza

Se analizaron los programas originales y se modificaron para reunir dos temperaturas por proceso y calcular el consumo total, máximo y mínimo. Ambos casos utilizan cuatro procesos; el rank 0 representa a la Oficina Central.

Caso 1 Parámetros de MPI Gather en el programa original

```
MPI_Gather(&temperatura, 1, MPI_FLOAT,  
          temperaturas, 1, MPI_FLOAT, 0, MPI_COMM_WORLD);
```

Parámetro	Valor original	Qué representa
sendbuf	&temperatura	Dirección de la medición local que aporta cada proceso.
sendcount	1	Una temperatura enviada por proceso.
sendtype	MPI_FLOAT	Tipo MPI correspondiente a los valores float.
recvbuf	temperaturas	Arreglo receptor de cuatro valores, utilizado en rank 0.
recvcount	1	Un valor recibido de cada proceso; no es el total de valores.
recvtype	MPI_FLOAT	Tipo de cada elemento recibido.
root	0	Rank que reúne las temperaturas, incluida su propia medición.
communicator	MPI_COMM_WORLD	Comunicador que contiene los cuatro procesos.

Modificación para dos mediciones

La variable local pasa a ser `float temperatura[2]` y el arreglo receptor pasa a tener ocho posiciones: `float temperaturas[8]`. Se conservan los tipos `MPI_FLOAT` y el `root 0`. Una sola llamada recibe los dos valores de cada proceso.

```
MPI_Gather(temperatura, 2, MPI_FLOAT,  
          temperaturas, 2, MPI_FLOAT, 0, MPI_COMM_WORLD);
```

La dirección del arreglo se pasa como `temperatura`, sin el operador `&` usado para la variable escalar original. El programa sigue exigiendo exactamente cuatro procesos para respetar el tamaño de los arreglos.

Caso 1 Análisis de los resultados

Por qué Gather conserva los resultados individuales

MPI_Gather copia los valores de cada proceso en el arreglo receptor sin aplicar una operación aritmética. Así, la Oficina Central puede consultar cada medición por separado, en lugar de obtener solamente un total o un promedio.

Relación entre la posición y el rank

Los bloques se almacenan en orden de rank. Con un dato por proceso, temperaturas[i] corresponde al rank i. Con dos datos, el rank i ocupa temperaturas[2 * i] y temperaturas[2 * i + 1], aunque sus mensajes en la terminal aparezcan en otro orden.

Rank	Posiciones en el arreglo	Temperaturas recibidas
0	0 y 1	24.5 y 25.0 °C
1	2 y 3	26.1 y 26.4 °C
2	4 y 5	23.8 y 24.1 °C
3	6 y 7	27.0 y 27.3 °C

Cambios necesarios al enviar dos valores

sendcount y recvcount cambian de 1 a 2. El búfer local necesita dos float y el receptor necesita $4 \times 2 = 8$ float. El ciclo de impresión también cambia para leer cada par. recvcount vale 2 porque expresa lo recibido de un proceso, no los ocho elementos reunidos.

Participación de los procesos

La llamada a Gather está fuera del bloque if (rank == 0): los cuatro procesos deben participar. El bloque exclusivo de rank 0 se usa para mostrar el arreglo reunido. En los otros procesos el contenido del búfer receptor no se utiliza.

Identificación e inicialización

MPI_Init inicia el entorno; MPI_Comm_rank obtiene el identificador de cada proceso; MPI_Comm_size permite comprobar que participan cuatro; MPI_Finalize cierra el entorno. Si la cantidad es distinta, todos finalizan antes de llegar a la comunicación y la versión modificada devuelve un código de error.

Caso 2 Consumo eléctrico con MPI Reduce

```
MPI_Reduce(&consumo, &consumo_total, 1, MPI_INT,  
          MPI_SUM, 0, MPI_COMM_WORLD);
```

Parámetro	Valor original	Qué representa
sendbuf	&consumo	Dirección del consumo local aportado por cada proceso.
recvbuf	&consumo_total	Dirección del resultado combinado, válido en rank 0.
count	1	Un entero aportado por proceso y un entero de resultado.
datatype	MPI_INT	Tipo MPI correspondiente a consumo, que es int.
op	MPI_SUM	Operación que suma los consumos.
root	0	Rank que recibe el resultado.
communicator	MPI_COMM_WORLD	Comunicador con todos los procesos participantes.

Modificación realizada

Se mantuvo la reducción de suma y se agregaron dos variables receptoras y dos llamadas: una con MPI_MAX y otra con MPI_MIN. Los cuatro procesos ejecutan las tres llamadas en el mismo orden. Solo rank 0 imprime los resultados. En C, los nombres correctos de las constantes son MPI_MAX y MPI_MIN.

Qué función cumple op

Indica cómo se combinan los datos aportados por los procesos. En este caso, los consumos son 150, 120, 180 y 100 kWh; se usa la misma entrada para las tres operaciones.

Operación	Cálculo o selección	Resultado
MPI_SUM	$150 + 120 + 180 + 100$	550 kWh
MPI_MAX	Mayor consumo entre los cuatro valores	180 kWh
MPI_MIN	Menor consumo entre los cuatro valores	100 kWh

Diferencia conceptual entre Gather y Reduce

Gather reúne y conserva los valores individuales. Reduce los combina según op y entrega el resultado al root. Aquí Gather produce ocho temperaturas; cada Reduce produce un entero. El count de Reduce es 1 porque cada proceso aporta un valor, aunque participen cuatro procesos.

Compilación y ejecución de los originales

Capturas reales de una terminal xterm en Linux. Los originales se compilaron sin modificaciones. Cada ejecución utilizó cuatro procesos. Las versiones del compilador y de Open MPI se muestran en la primera captura.

Ejercicio1a original

```
$ mpicc --version
$ head -n 1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
$ mpirun --version
$ head -n 1
mpirun (Open MPI) 4.1.6
$ mpicc -std=c11 -Wall -Wextra -Wpedantic -Werror ../upload/Ejercicio1a.c -o bin/original_a
$ mpirun --oversubscribe -np 4 ./bin/original_a
Proceso 0: temperatura registrada = 24.5 C

Oficina Central: temperaturas recibidas
Proceso 0: 24.5 C
Proceso 1: 26.1 C
Proceso 2: 23.8 C
Proceso 3: 27.0 C
Proceso 1: temperatura registrada = 26.1 C
Proceso 2: temperatura registrada = 23.8 C
Proceso 3: temperatura registrada = 27.0 C
$ printf 'Estado de salida: %d\n' 0
Estado de salida: 0
$ set +x
```

La Oficina Central recibe cuatro temperaturas: 24.5, 26.1, 23.8 y 27.0 °C.

Ejercicio1b original

```
$ mpicc -std=c11 -Wall -Wextra -Wpedantic -Werror ../upload/Ejercicio1b.c -o bin/original_b
$ mpirun --oversubscribe -np 4 ./bin/original_b
Proceso 3: consumo = 100 kWh
Proceso 0: consumo = 150 kWh

Consumo total: 550 kWh
Proceso 1: consumo = 120 kWh
Proceso 2: consumo = 180 kWh
$ printf 'Estado de salida: %d\n' 0
Estado de salida: 0
$ set +x
```

La suma de los consumos originales es 550 kWh.

Compilación y ejecución de los modificados

Se compiló con C11 y las opciones -Wall -Wextra -Wpedantic -Werror. La compilación terminó sin advertencias ni errores. Las capturas registran comandos, resultados y estado de salida.

Ejercicio1a con dos mediciones por proceso

```
$ mpicc -std=c11 -Wall -Wextra -Wpedantic -Werror Ejerciciola.c -o bin/Ejerciciola
$ mpirun --oversubscribe -np 4 ./bin/Ejerciciola
Proceso 0: temperaturas registradas = 24.5 C, 25.0 C

Proceso 1: temperaturas registradas = 26.1 C, 26.4 C
Proceso 2: temperaturas registradas = 23.8 C, 24.1 C
Proceso 3: temperaturas registradas = 27.0 C, 27.3 C
Oficina Central: temperaturas recibidas
Proceso 0: 24.5 C, 25.0 C
Proceso 1: 26.1 C, 26.4 C
Proceso 2: 23.8 C, 24.1 C
Proceso 3: 27.0 C, 27.3 C
$ printf 'Estado de salida: %d\n' 0
Estado de salida: 0
$ set +x
█
```

Se recibieron los ocho valores en el orden de rank indicado en el análisis.

Ejercicio1b con suma máximo y mínimo

```
$ mpicc -std=c11 -Wall -Wextra -Wpedantic -Werror Ejerciciolb.c -o bin/Ejerciciolb
$ mpirun --oversubscribe -np 4 ./bin/Ejerciciolb
Proceso 1: consumo = 120 kWh
Proceso 2: consumo = 180 kWh
Proceso 3: consumo = 100 kWh
Proceso 0: consumo = 150 kWh

Consumo total: 550 kWh
Consumo maximo: 180 kWh
Consumo minimo: 100 kWh
$ printf 'Estado de salida: %d\n' 0
Estado de salida: 0
$ set +x
█
```

Los resultados coinciden con el cálculo: 550 kWh en total, 180 kWh como máximo y 100 kWh como mínimo. Las pruebas adicionales con uno y tres procesos rechazaron correctamente la ejecución y finalizaron sin bloqueo.

Reproducción local y referencias

Archivos del ejercicio y video de explicación

Los archivos del ejercicio están disponibles en el repositorio de GitHub:

<https://github.com/lfmendoza/cc3069-ejercicio-openmpi>

El enlace al video de explicación se publicará en la sección Video de explicación del README.md del mismo repositorio, al finalizar la grabación. Esta sección puede actualizarse sin modificar el enlace entregado en este documento.

Reproducción en Ubuntu o WSL2

```
bash ejecutar.sh
```

Ejecutar este comando desde la carpeta extraída del ZIP. El script compila ambos archivos y guarda el registro en evidencias/ejecucion.txt. Requiere Open MPI y GCC. Las capturas de este informe corresponden al entorno Linux de preparación, no a una ejecución en la computadora del estudiante.

Alcance de la hoja proporcionada

La hoja desarrolla los casos 1a y 1b, aunque el apartado de entrega menciona tres ejercicios. No se adjuntó un tercer enunciado. Tampoco se solicita implementar Send, Recv, Bcast o Scatter en estos casos. Se resolvieron las modificaciones y todas las preguntas incluidas.

Referencias

Hoja y códigos originales proporcionados para el ejercicio. Documentación oficial consultada para el orden de recepción y las operaciones colectivas:

[Open MPI - MPI_Gather](#)

[Open MPI - MPI_Reduce](#)